

Multithreading Project in C

Thread Pool with Producer–Consumer Model (POSIX Threads)



Pankaj Kumar Chaudhary
MSC'27

Department of Computer Science, University of Delhi

1. Introduction

This project demonstrates how multithreading works in C using the POSIX threads (pthread) library. It combines two important concepts:

- Producer–Consumer problem
- Thread Pool

The system creates a shared queue where tasks are added and a fixed number of worker threads process those tasks. Proper synchronization is used to avoid race conditions and ensure safe execution.

2. Objectives

- Understand how threads are created and managed
- Learn how race conditions occur and how to fix them
- Implement a shared queue safely
- Use mutex and condition variables for synchronization
- Build a thread pool for efficient task execution

3. Tools and Technologies

- Language: C
- Library: pthread (POSIX Threads)
- Compiler: gcc
- Environment: Linux / WSL (Ubuntu)

4. Core Concepts

Multithreading

Multiple threads running within the same program.

Race Condition

Occurs when multiple threads access shared data at the same time.

Mutex

A lock that ensures only one thread accesses critical code at a time.

Condition Variables

Allow threads to wait and get notified when a condition changes.

Producer–Consumer

Producer adds tasks, consumer processes tasks.

Thread Pool

A fixed number of threads reused for handling multiple tasks.

5. Project Structure

Multi-threading/

|

|— main.c

|— queue.c

|— queue.h
|— thread_pool.c
|— thread_pool.h

6. Code Explanation

6.1 Queue Definition (queue.h)

```
#ifndef QUEUE_H
#define QUEUE_H

#include <pthread.h>

#define MAX 5 // maximum size of queue

// Structure representing a circular queue
typedef struct {
    int buffer[MAX]; // array to store tasks
    int front, rear; // front for removal, rear for insertion
    int count; // current number of items

    pthread_mutex_t lock; // mutex to protect shared queue
    pthread_cond_t not_full; // condition when queue is not full
    pthread_cond_t not_empty; // condition when queue is not empty
} Queue;

// function declarations
```

```
void init_queue(Queue *q);  
void enqueue(Queue *q, int item);  
int dequeue(Queue *q);
```

```
#endif
```

6.2 Queue Implementation (queue.c)

Initialization

```
#include "queue.h"  
  
#include <pthread.h>  
  
// Initialize queue values and synchronization primitives  
void init_queue(Queue *q) {  
    q->front = 0;  
    q->rear = 0;  
    q->count = 0;  
  
    // initialize mutex and condition variables  
    pthread_mutex_init(&q->lock, NULL);  
    pthread_cond_init(&q->not_full, NULL);  
    pthread_cond_init(&q->not_empty, NULL);  
}
```

Enqueue (Producer)

```
void enqueue(Queue *q, int item) {

    // lock the queue before modifying shared data
    pthread_mutex_lock(&q->lock);

    // if queue is full, wait until space becomes available
    while(q->count == MAX) {
        // releases lock and waits
        pthread_cond_wait(&q->not_full, &q->lock);
    }

    // insert item at rear
    q->buffer[q->rear] = item;

    // update rear index in circular manner
    q->rear = (q->rear + 1) % MAX;

    // increase count
    q->count++;

    // notify consumers that queue is not empty anymore
    pthread_cond_signal(&q->not_empty);

    // unlock the queue
```

```
    pthread_mutex_unlock(&q->lock);  
}
```

Dequeue (Consumer)

```
int dequeue(Queue *q) {  
  
    // lock queue before accessing  
    pthread_mutex_lock(&q->lock);  
  
    // if queue is empty, wait for producer  
    while(q->count == 0) {  
        pthread_cond_wait(&q->not_empty, &q->lock);  
    }  
  
    // get item from front  
    int item = q->buffer[q->front];  
  
    // update front index  
    q->front = (q->front + 1) % MAX;  
  
    // decrease count  
    q->count--;  
  
    // notify producer that space is available  
    pthread_cond_signal(&q->not_full);  
}
```

```
// unlock queue

pthread_mutex_unlock(&q->lock);

return item;
}
```

6.3 Thread Pool Definition (thread_pool.h)

```
#ifndef THREAD_POOL_H
#define THREAD_POOL_H

#include <pthread.h>
#include "queue.h"

#define NUM_THREADS 3 // number of worker threads

// Thread pool structure
typedef struct {
    pthread_t threads[NUM_THREADS]; // worker threads
    Queue *queue; // shared task queue
    int stop; // flag to stop threads
} ThreadPool;

void init_thread_pool(ThreadPool *pool, Queue *queue);
```

```
void destroy_thread_pool(ThreadPool *pool);
```

```
#endif
```

6.4 Thread Pool Implementation (thread_pool.c)

Worker Thread

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include "thread_pool.h"
```

```
// function executed by each worker thread
```

```
void* worker(void* arg) {
```

```
    // convert argument to ThreadPool pointer
```

```
    ThreadPool *pool = (ThreadPool*)arg;
```

```
    // infinite loop so thread keeps working
```

```
    while(1) {
```

```
        // lock queue before accessing shared data
```

```
        pthread_mutex_lock(&pool->queue->lock);
```

```
        // wait if queue is empty and stop signal not given
```



```
while(pool->queue->count == 0 && !pool->stop) {  
    // releases lock and waits for signal  
    pthread_cond_wait(&pool->queue->not_empty, &pool->queue->lock);  
}
```

```
// if stop flag is set, exit thread  
if(pool->stop) {  
    pthread_mutex_unlock(&pool->queue->lock);  
    break;  
}
```

```
// remove task from queue  
int task = pool->queue->buffer[pool->queue->front];
```

```
// update front index  
pool->queue->front = (pool->queue->front + 1) % MAX;
```

```
// decrease count  
pool->queue->count--;
```

```
// notify producer that space is available  
pthread_cond_signal(&pool->queue->not_full);
```

```
// unlock queue
```

```

pthread_mutex_unlock(&pool->queue->lock);

// process task outside critical section
printf("Thread %lu processing task %d\n", pthread_self(), task);

// simulate some work
sleep(1);
}

return NULL;
}

```

Initialize Thread Pool

```

void init_thread_pool(ThreadPool *pool, Queue *queue) {

// assign queue to pool
pool->queue = queue;

// initially stop is false
pool->stop = 0;

// create worker threads
for(int i = 0; i < NUM_THREADS; i++) {
    pthread_create(&pool->threads[i], NULL, worker, pool);
}
}

```

```
}  
}
```

Destroy Thread Pool

```
void destroy_thread_pool(ThreadPool *pool) {  
  
    // lock queue before modifying stop flag  
    pthread_mutex_lock(&pool->queue->lock);  
  
    // set stop flag  
    pool->stop = 1;  
  
    // wake all waiting threads  
    pthread_cond_broadcast(&pool->queue->not_empty);  
  
    // unlock queue  
    pthread_mutex_unlock(&pool->queue->lock);  
  
    // wait for all threads to finish  
    for(int i = 0; i < NUM_THREADS; i++) {  
        pthread_join(pool->threads[i], NULL);  
    }  
}
```

6.5 Main Program (main.c)

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include "queue.h"
```

```
#include "thread_pool.h"
```

```
Queue q;
```

```
ThreadPool pool;
```

```
int main() {
```

```
    // initialize queue
```

```
    init_queue(&q);
```

```
    // initialize thread pool
```

```
    init_thread_pool(&pool, &q);
```

```
    // add tasks to queue (acts as producer)
```

```
    for(int i = 1; i <= 10; i++) {
```

```
        enqueue(&q, i);
```

```
    }
```

```
    // allow some time for processing
```

```
    sleep(5);
```

```
// shutdown thread pool

destroy_thread_pool(&pool);

return 0;

}
```

7. Execution Flow

1. Program starts
2. Queue is initialized
3. Thread pool is created
4. Worker threads start and wait for tasks
5. Main thread adds tasks to queue
6. Workers wake up and process tasks
7. After tasks are done, stop signal is sent
8. All threads exit safely

8. Key Features

- Thread-safe queue implementation
- Proper synchronization using mutex
- Efficient waiting using condition variables
- No busy waiting
- Clean thread shutdown mechanism

9. Compilation and Execution

```
gcc main.c queue.c thread_pool.c -o program -pthread
```

```
./program
```

10. Sample Output

```

pankaj@DESKTOP-GK8CFVR:/mnt/c/Users/Dell/learning_c/Multi-threading$ gcc main.c queue.c thread_pool.c -o program -pthread
pankaj@DESKTOP-GK8CFVR:/mnt/c/Users/Dell/learning_c/Multi-threading$ ./program
Produced: 1
Thread 132218557036224 processing task 1
Produced: 2
Thread 132218548643520 processing task 2
Produced: 3
Thread 132218540250816 processing task 3
Produced: 4
Thread 132218557036224 processing task 4
Produced: 5
Thread 132218548643520 processing task 5
Produced: 6
Thread 132218540250816 processing task 6
Produced: 7
Thread 132218540250816 processing task 7
Produced: 8
Thread 132218540250816 processing task 8
Produced: 9
Thread 132218557036224 processing task 9
Produced: 10
Thread 132218548643520 processing task 10
pankaj@DESKTOP-GK8CFVR:/mnt/c/Users/Dell/learning_c/Multi-threading$ |

```

11. Advantages

- Reduces overhead of creating threads repeatedly
- Efficient CPU usage
- Avoids race conditions
- Scalable design

12. Applications

- Web servers
- Operating systems
- Embedded systems
- Task schedulers
- Network request processing

13. Conclusion

This project demonstrates how multithreading can be implemented effectively in C using pthreads. It combines synchronization techniques with a thread pool design to create a system that is both efficient and safe. The concepts used here are directly applicable in real-world systems.

Thank You